

TD4-16801

TD4-17020

- Individual passwords for each logger

TD4-17282

Type: User Story	Area:	Due Date:
Severity: Should Have	Feature:	Estimate in Story Points:
Author: Schmidt, Stefan	Assignee(s): Ashok, Nishitha	Initial Estimate: 9d
Project: IPEmotionRT	Status: Implemented	Initial Estimate Dev: 5d
Target Project:	Resolution:	Initial Estimate Testing: 4d
RN-Text: Individual password for each logger	Priority: Prio 2 Must [80.0]	Remaining Estimate: 5d 5 3/4h
RN-Position: 20	Planned In: Sprint_2026-19 (2026-10-01)	Time Spent: 3d 2 1/4h
AI: AI Support	Target Release: V09.02.00 - 2026 R3	Dev-Status:
AI Savings: 1d	Unplanned:	Legacy:

Discontinued Fields

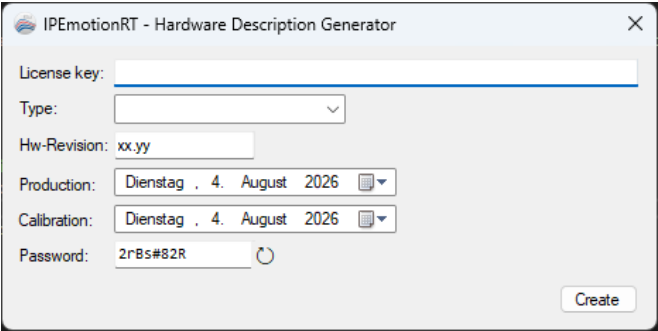
IpeRole:
Tag:
Backlog: Product_Backlog

Description

As PO of IPEmotionRT I expect that the hardware description generator will be able to create an individual ssh password for the user account. This password is encrypted via md5checksum or similar and needs to be shown without encryption in the dialog, so that the testfield can know it. The password will be added to the hw_descr.xml
RT will need to read this password from the hw_descr.xml and use it in case no user defined password is set.
RT versions < 2026 R3 will still use user-user by default.
hw_descr files without the entry still need to work (not change at runtime).

Acceptance criterion:
- documentation
- unit test

Documentation:
The Hardware Description Generator tool now creates an individual password for the user account during hardware description creation. The password is generated randomly, shown once in the dialog so it can be noted down, and stored in hw_descr.xml only as an SHA256 crypt hash, never as plain text on disk.



1. A password generator creates an eight character random password.
The character set excludes visually ambiguous characters such as zero, capital O, one, lowercase l and capital I. A small set of special characters that are reachable on both German and US keyboard layouts without the AltGr key is included by design. So operators can read and type the password without confusion.
2. A SHA256 crypt implementation was added, following the SHA256/SHA512 crypt specification originally published by Ulrich Drepper and used by glibc and busybox chpasswd. It produces the same hash format the logger already understands (the dollar five dollar prefix format), so no changes were required on the busybox chpasswd side. The implementation was verified against the seven official reference test vectors from that specification and additionally verified end to end against a physical logger (Logger 00536): a generated hash was written to shadow using chpasswd with the encrypted flag, and login with the corresponding plain text password succeeded.
3. Full unit test coverage (28 NUnit tests, 100 percent statement coverage of Sha256Crypt as measured with dotCover) covers the official test vectors, argument validation, salt handling including trailing suffix truncation, and the password generator character set and length behaviour.
4. The generator dialog now shows the generated password in a read only field with a refresh button to generate a new one, plus tooltips explaining the purpose of the field. When the Create button is used, the tool computes the hash and writes it into hw_descr.xml as a new element named UserPassword, then shows a confirmation dialog with the plain text password for the operator to record.

The new XML element is a flat child of the HardwareDescription root element, named UserPassword, and contains only the crypt hash string, for example dollar five dollar followed by a sixteen character salt, a dollar sign, and the hash:

```
<?xml version="1.0" encoding="utf-8"?>
<HardwareDescription>
...
<UserPassword>$5$CK6rQScDpvOxuypi$0aZMd1la068z1I00EZIkWu4F/oD3NeS60ohziEBrS43</UserPassword>
```

</HardwareDescription>

hw_descr.xml files that do not contain this element remain valid and unaffected.

Background on Linux password storage:

On Linux, a password is never stored in plain text. The system only stores a hash of the password in the file `/etc/shadow`, computed by a one way function called `crypt`. A random salt is mixed into the hash together with the password, so the same password produces a different looking hash every time it is set, and precomputed hash tables cannot be used to look up a password from its hash.

The `crypt` function supports several algorithm families, identified by a prefix in the stored hash string. No prefix at all denotes the original algorithm from the 1970s, based on DES, limited to a two character salt and only the first eight characters of the password, any characters beyond that are silently ignored. The prefix dollar one denotes MD5, dollar five dollar denotes SHA256, and dollar six dollar denotes SHA512, all of which support a much longer salt and use the full password.

Dollar five dollar, SHA256, was chosen as the format for the `UserPassword` element in `hw_descr.xml` for three reasons. First, the original DES algorithm silently ignores password characters beyond the eighth, which would make part of a generated password meaningless without any visible warning. Second, SHA256 was already in active use on the logger, an account named `rtssupport` was found to already use it. Third, it was directly confirmed on a physical logger that both `busybox chpasswd`, used to apply the hash, and the actual login mechanism accept and verify it correctly.

Stronger modern algorithms such as `bcrypt` or `Argon2` were not considered, because the `busybox` build on this platform only offers DES, MD5, SHA256 and SHA512 through `chpasswd` and `mkpasswd`, and the login mechanism only ever verifies whichever of these the underlying C library `crypt` function supports. Anything outside that fixed set would require changing the `busybox` or C library build itself, which was out of scope here.

Flow on the logger:

At every boot, before the SSH server is started, the boot script `S39EditUserPassword` decides which password to apply to the operating system account named `user`. It reads `Runtime.xml`, which the logger updates whenever a configuration update from `IPEmotion` is applied, and checks the access restriction password field first. If that field holds a value, it is applied directly with `chpasswd` using the encrypted flag, since it is never a default value.

If no access restriction password is set, the script checks the SSH password field in `Runtime.xml` instead. Because that field defaults to the literal value `user` and is exported on nearly every configuration update, its mere presence does not prove that a customer intentionally chose it. The script therefore recomputes the `crypt` hash of the literal value `user`, using `mkpasswd` with the same algorithm and salt as the stored value, and compares the result to it. A match means the stored value is still the unmodified factory default, a mismatch means a genuine custom password was configured on purpose and continues to be used.

Only when the SSH password field is empty, or found to still be the unmodified default, does the script read the `UserPassword` element from `hw_descr.xml`, produced once by the Hardware Description Generator during manufacturing, and apply that hash instead. If none of the above yields a password, the account keeps whatever password was already active, which is the `user` and `user` default on a logger that has not received any configuration update yet.

Password priority:

The user account password on the logger can currently be influenced from three places:

- Options, Logger, Security, Access restriction
When Use access restriction is enabled, this password becomes the account password, any password entered under SSH is ignored and never even sent to the logger.
- Options, Logger, Security, SSH
This password only takes effect when Use access restriction is disabled.
When enabled, it becomes the account password on the next configuration update, overriding whatever was set before.
- `hw_descr.xml`, `UserPassword` element, produced by the Hardware Description Generator.
This has the lowest priority.
It is only used when no access restriction password is set, and the SSH password is either not set or found to still be the unmodified factory default.

There is only one operating system account, named `user`, with one active password at any time. The two settings above are not combined or merged. Depending on which one was configured during the most recent configuration update, that one becomes the active password. The `Runtime.xml` file on the logger keeps two separate memory fields, one for the access restriction password and one for the SSH password, reflecting which mechanism was used last, but only one of them corresponds to the password that is actually active. The SSH password field in the Options dialog defaults to the literal value `user` and cannot be left empty in the user interface, so it is exported and applied on nearly every configuration update for which access restriction is disabled, whether or not the customer ever intentionally set a custom SSH password.

Because of this default value, a fallback rule on the logger that only checks whether the SSH password field in `Runtime.xml` is empty would almost never apply, since that field is populated on nearly every configuration update regardless of customer intent. Instead, the boot script recomputes the `crypt` hash of the literal value `user`, using the same algorithm and salt as the value already stored in `Runtime.xml`, with the `mkpasswd` utility that is already present on the logger, and compares the result to the stored value. If they match, the stored password is still the unmodified factory default and the boot script uses the individual password from `hw_descr.xml` instead. If they do not match, a genuine custom password was configured on purpose and it continues to be used. This check is skipped whenever the access restriction password field is set, since that value is never a default and always takes priority. One accepted limitation of this approach is that a customer who deliberately sets the SSH password back to the literal value `user` cannot be distinguished from a password that was simply never touched, so `hw_descr.xml` would still take over in that case.

Config update acceptance rule:

Once a device has a `UserPassword`, an incoming `hw_descr.xml` is only accepted if it carries the exact same `UserPassword` value, in addition to the existing `SerialNumber` check. If either check fails, `hw_descr.xml` alone is removed from the update package, exactly like an existing `SerialNumber` mismatch, and the rest of the update proceeds normally.

Unit Tests:

Every new line of code introduced for this feature, `PasswordGenerator`, `Sha256Crypt` and `HardwareDescriptionXmlBuilder`, is fully covered by unit tests. The current NUnit test project contains 34 tests and measures 100 percent statement coverage on all three of these classes, as shown by `dotCover`. This includes the official SHA256 `crypt` reference test vectors, argument validation, salt handling, and the password character set and length behaviour.

Symbol	Coverage (%) ▲	Uncovered/Total Stmts.
📁 Total	61%	192/498
📁 IpemotionRt.HardwareDescriptionGenerator	51%	192/392
🔗 IpemotionRt.HardwareDescriptionGenerator	51%	192/392
▶ 📁 Program	0%	5/5
▶ 📁 Form1	0%	187/187
▶ 📁 HardwareDescriptionXmlBuilder	100%	0/13
▶ 📁 PasswordGenerator	100%	0/39
▶ 📁 Sha256Crypt	100%	0/148
📁 IpemotionRt.HardwareDescriptionGenerator.Tests	100%	0/106
🔗 IpemotionRt.HardwareDescriptionGenerator.Tests	100%	0/106
▶ 📁 HardwareDescriptionXmlBuilderTest	100%	0/25
▶ 📁 PasswordGeneratorTest	100%	0/40
▶ 📁 Sha256CryptTest	100%	0/41

Form1 and Program remain at 0 percent coverage, and this is expected rather than an oversight. Graphical user interface code in general is difficult to cover with automated unit tests. Form1 opens a native SaveFileDialog and shows MessageBox windows, both of which block execution and wait for a person to click something, so a test that calls into Form1 directly would simply hang rather than complete. Program only contains the single line that starts the application and has no logic of its own to test.

To keep the testable logic separate from this problem, the actual decision making, building the hw_descr.xml content and computing the password hash, was moved out of Form1 and into HardwareDescriptionXmlBuilder, which has no dependency on any user interface element and is fully covered as a result. Form1 itself was intentionally left untested, since doing so would require introducing test only abstractions around the dialogs for a small internal tool, which was judged not to be worth the additional complexity.

Four new Test cases were added to UnitTest TestJobFrameworkMain.cpp, covering acceptance, rejection on mismatch, first time adoption, and removal from the irf archive on rejection. All four were also verified end to end on a physical logger (Logger 00536), together with a full regression run of the existing JobFrameWork test suite.

Linked Work Items

Suspect	Role	Title	Project	Revision	Status	Assignee(s)
	implements	🏠 TD4-16801 - Concept/Research: Individual passwords for each user	IPemotionRT		🔧	Ashok, Nishitha
	is verified by	🏠 TD4-17282 - GUI test for individual passwords for each user	IPemotionRT		🔧	

Linked Revisions

Revision	Message	Created	Author
📄 71665	TD4-17020 Add individual user pass...	2026-08-04 16:50	Grundeir, Lars
📄 71666	TD4-17020 updated ignore lists	2026-08-04 16:53	Grundeir, Lars
📄 71667	TD4-17020 set TargetFrameworkVer...	2026-08-04 16:55	Grundeir, Lars
📄 71669	TD4-17020 updated ignore lists	2026-08-04 16:59	Grundeir, Lars
📄 71669	TD4-17020 updated ignore lists	2026-08-04 16:59	Grundeir, Lars
📄 99058	TD4-17020 updated ignore lists	2026-08-04 17:00	Grundeir, Lars
📄 71670	TD4-17020 Add XML doc comments...	2026-08-04 19:26	Grundeir, Lars
📄 71725	TD4-17020: Preserve UserPassword i...	2026-08-07 19:39	Grundeir, Lars
📄 71734	TD4-17020: Require matching UserP...	2026-08-10 14:08	Grundeir, Lars
📄 71808	TD4-17020 updated Release/Ipemot...	2026-08-17 09:24	Grundeir, Lars
📄 71809	TD4-17020 updated Release/Ipemot...	2026-08-17 09:26	Grundeir, Lars
📄 71810	TD4-17020 removed messagebox af...	2026-08-17 09:44	Grundeir, Lars
📄 71811	TD4-17020 updated Release/Ipemot...	2026-08-17 09:44	Grundeir, Lars
📄 71812	TD4-17020 translate german tooltip...	2026-08-17 09:49	Grundeir, Lars
📄 71813	TD4-17020 updated Release/Ipemot...	2026-08-17 09:50	Grundeir, Lars

Work Records

User	Date	Time Spent	Type	Comment
Grundeir, Lars	2026-08-03	6h	Development	
Grundeir, Lars	2026-08-04	7 3/4h	Development	
Grundeir, Lars	2026-08-05	1 3/4h	Development	

9/21/26, 5:35 PM

polarion.ad.ipetronik.com/polarion/print/TESTdrive4/TD4-17020?filter=false&uiRole=testDesigner

Grunde, Lars	2026-08-07	4 3/4h	Development
Grunde, Lars	2026-08-10	5 3/4h	Development
Grunde, Lars	2026-08-17	1/4h	Development

Hyperlinks

Role	URL	Actions
internal reference	http://polarion.ad.ipetronik.com/polarion/#/project/TESTdrive4/wiki/CreateTasksForUserStory?SelectedUserStory=TD4-17020	
internal reference	http://polarion.ad.ipetronik.com/polarion/#/project/TESTdrive4/wiki/CreateTaskForUserStoryInPlans?SingleReq=TD4-17020	

Attachments

Title	File Name	Size	Author	Last Modified	Actions
screenshot-20260804-170644	screenshot-20260804-170644.png	14881 B	Grunde, Lars	2026-08-04 17:06	Show revisions
screenshot-20260804-195738	screenshot-20260804-195738.png	33916 B	Grunde, Lars	2026-08-04 19:57	Show revisions

Created: 2026-05-27 10:53 , Updated: 2026-09-17 11:31

Powered by Polarion® ALM™ 22 R2 (Build: 3.22.2.20221019-1425-22_R2-8fd9e182)